Replicated Counter Service: Test Report

Akylbek Askhat Serikuly

Digital Engineering, Narxoz University

Distributed Systems

Assignment (Labs 2-5) and Midterm Defense

9 October 2026

Repository: https://github.com/askhat06/MidtermDS

Replicated Counter Service: Test Report

The system is a counter service over gRPC. Part A is one server with duplicate suppression and a client with deadlines and retries. Part B adds Lamport clocks and an event log. Part C runs three independent replicas; the client sends each write to all three and reports it as committed only when at least two acknowledge. This report records how each behaviour was tested and what was observed. Every claim below points to a test name or to a captured log line.

RPC semantics (Task A2)

The client gives at-least-once delivery: every call has a 2 second deadline and on DEADLINE_EXCEEDED or UNAVAILABLE it is sent again, at most three more times, after pauses of 0.2, 0.4 and 0.8 s. The server gives at-most-once execution of the mutation: Increment looks up the idempotency key and applies the delta inside one lock, and a key that is already stored returns the stored result with was_duplicate = true. The key is created once per logical operation, before the first attempt, and every retry carries it. That is what makes a retry safe: the server can tell the same operation arriving again from a new operation. Without the key, a retry after a lost reply would add the delta a second time (tests T2 and T5).

1. Requirements under test

Table 1

Requirements Under Test and the Tests That Verify Them

Task	Requirement	Verified by
A2	A repeated idempotency key returns the stored result and is not applied again.	T2, F2
A2	Calls have a deadline; after a timeout the client retries with the same key and the counter moves exactly once.	T5, F3, fault log in section 4
A2	A single Increment applies its delta; Get on an unknown counter returns found = false.	T1, T4
A3	Two concurrent clients with 1,000 increments each give exactly 2,000 (8 worker threads, one lock around the dedup check and the mutation).	T3
B1	Lamport rules: +1 before a local event, max(own, received) + 1 on receive; the value is attached to every message; one log line per event in the required format.	Captured trace, Appendix A (checked line by line in section 6). No automated test.
C1	A write commits with two of three acknowledgements.	T6, F1, Figure 2
C1	A write with one acknowledgement is reported as failed and no value is presented as committed.	T7, Figure 2
C1	After a batch of quorum writes all replicas hold the same values.	T8

2. Test environment

Table 2
Test Environment

Item	Value
Operating system	Microsoft Windows 11 Home, version 10.0.26200 (build 26200), 64-bit
Machine	ASUS ROG Strix G614JI laptop, Intel 64 Family 6 Model 183 processor (about 2.6 GHz), 32 GB RAM
Python	CPython 3.14.6, virtual environment (.venv)
Packages	grpcio 1.84.0, grpcio-tools 1.84.0, protobuf 7.36.2, pytest 9.1.1 (full pip freeze in Appendix B)
How the tests run	pytest starts every server inside the test process on a free port chosen by the operating system and stops it when the test ends. Nothing has to be started by hand.
Commands	`pytest -v` and `python tests/perf_benchmark.py`

3. Test cases: expected and actual results

Result of the run: **11 passed in 16.20s**. The full pytest output is in Appendix C.

Table 3
Test Cases With Expected and Actual Results

Id	Test	Purpose	Expected result	Actual result	Result
T1	`test_increment_applies_delta`	A single Increment applies exactly its delta.	incr(x, 5): committed, new_value = 5, Get(x) = 5	committed, new_value = 5, Get(x) = 5	pass
T2	`test_duplicate_key_not_reapplied`	The same idempotency key sent twice is applied once.	second reply: new_value = 5 (not 10), was_duplicate = true; Get(x) = 5	second reply new_value = 5, was_duplicate = true; Get(x) = 5	pass
T3	`test_concurrent_increments_exact`	Two concurrent clients, 1,000 increments each.	Get(x) = 2000 exactly	Get(x) = 2000	pass
T4	`test_get_missing_counter`	Get on a counter that was never written.	found = false, no exception	found = false, no exception	pass
T5	`test_retry_after_timeout_is_safe`	First reply delayed 1000 ms, client deadline 300 ms; the client retries with the same key.	committed after the retry, was_duplicate = true, new_value = 5, Get(x) = 5	committed, was_duplicate = true, new_value = 5, Get(x) = 5	pass
T6	`test_majority_commit_two_acks`	Quorum write with replica C stopped.	committed, acks = 2	committed, acks = 2	pass
T7	`test_no_commit_below_majority`	Quorum write with replicas B and C stopped.	not committed, acks = 1, new_value = None	not committed, acks = 1, new_value = None	pass

Id	Test	Purpose	Expected result	Actual result	Result
T8	`test_replicas_converge`	40 quorum writes (20 to x by 1, 20 to y by 2), then read each replica.	every replica: x = 20, y = 40	every replica: x = 20, y = 40	pass
F1	`test_replica_crash_mid_request_still_commits`	Replica C is stopped while the write is waiting inside it.	incr() returns normally, committed, acks = 2	returned normally, committed, acks = 2	pass
F2	`test_duplicate_request_moves_value_once`	The same key is sent twice to three replicas.	second result was_duplicate = true, new_value = 5; every replica holds 5	was_duplicate = true, new_value = 5; every replica holds 5	pass
F3	`test_timeout_then_retry_moves_value_once`	Replica C answers 2000 ms late, client deadline 300 ms.	committed, acks = 3 after the retry; every replica holds 5	committed, acks = 3; every replica holds 5	pass

4. Failure-injection results

The faults are simulated inside the server (route b of Task C3). The flag `--fault delay --delay-ms N` makes a replica apply its first Increment and then wait N milliseconds before answering, so the reply arrives after the client's deadline. A crash is simulated by stopping a replica.

Table 4
Failure-Injection Scenarios and Observed Invariants

Scenario	Injected fault	Observed invariant	Evidence
Replica crash mid-request	Replica C holds the request for 2000 ms; it is stopped 0.5 s after the client started the write.	The client still commits with the two remaining acknowledgements and no exception reaches the caller.	F1: PASSED
Request duplication	The same idempotency key is sent twice to three replicas.	The second response carries was_duplicate = true and every replica still holds 5.	F2: PASSED
Induced timeout with retry	Replica C answers 2000 ms late; the client deadline is 300 ms.	The retry succeeds (3 of 3 acknowledgements) and the counter is 5 on every replica: it moved exactly once.	F3: PASSED, T5: PASSED, log excerpt below

Log evidence for the third scenario, captured by hand with one replica started with `--fault delay --delay-ms 3000` and the default 2 second client deadline. The client logged 2 SEND lines for one logical operation: the first attempt timed out and was sent again with the same key. The replica logged 2 RECV lines but only 1 APPLY line: the second request was answered from the deduplication store. The client printed `OK committed value=5 (replicas acked: 1/1, duplicate: yes).`

```
[client-1] SEND       Increment(counter=x, delta=5)                    L=1
[client-1] SEND       Increment(counter=x, delta=5)                    L=2
[client-1] RECV       IncrementReply(new_value=5)                      L=6   (received L=5)

[replica-A] RECV      Increment(counter=x, delta=5)                    L=2   (received L=1)
```

```
[replica-A] APPLY     counter=x -> 5                                L=3
[replica-A] RECV      Increment(counter=x, delta=5)                 L=4   (received L=2)
[replica-A] SEND      IncrementReply(new_value=5)                   L=5
[replica-A] SEND      IncrementReply(new_value=5)                   L=6
```

5. Performance

Method: `tests/perf_benchmark.py` starts three replicas, sends 2,000 Increment requests per configuration and measures each call on the client with time.perf_counter(), from just before incr() to just after it returns. The median is statistics.median; p95 is the value at position ceil(0.95 * n) of the sorted latencies, counted from one. Event logging is switched off during the measurement.

Table 5

Increment Latency by Configuration

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.30	0.39	2000
Single replica, 16 clients	3.69	4.65	2000
Quorum (3 replicas), 1 client	0.91	1.20	2000
Quorum (3 replicas), 16 clients	11.39	13.82	2000

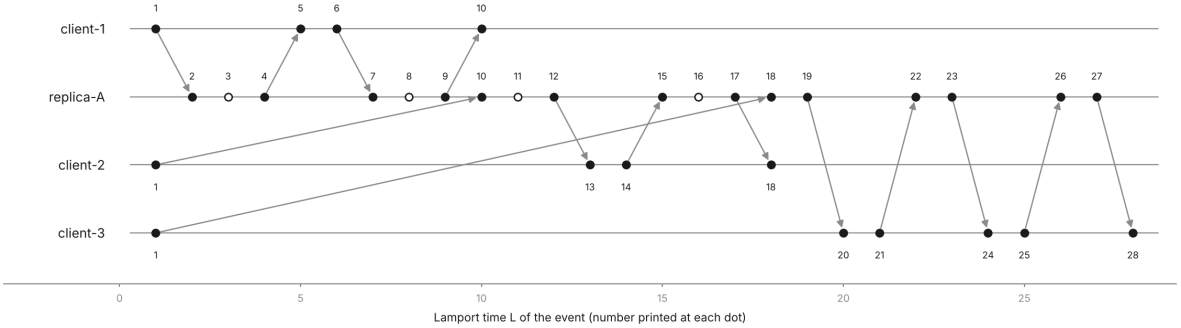
With one client a quorum write takes about 3.0 times as long as a write to a single replica (median 0.91 ms against 0.30 ms). The extra time comes from the fan-out: the client calls the three replicas one after another, so one write costs three network round trips and three lock acquisitions instead of one. With 16 clients the latency grows in both paths (median 3.69 ms and 11.39 ms) because requests queue behind the 8 worker threads of each server and, in this benchmark, behind Python's global interpreter lock, since the clients and the replicas run in one process. For a like counter this cost is acceptable: even the slowest configuration has a p95 of 13.82 ms, and in exchange a committed write survives the loss of one replica. Sending the three requests in parallel would bring the quorum latency close to that of the slowest single replica.

6. Ordering analysis of the Lamport trace (Task B2)

Scenario: one replica; client-1 increments x twice, client-2 increments y twice, client-3 issues three Get calls. Each process wrote its own log file; the 32 lines are in `logs/` and in Appendix A. In this run the three clients were started one after another from one terminal, so in wall-clock time they did not overlap. The logical relations below do not depend on that.

Figure 1

Happened-Before Diagram of the Captured Trace



Note. Each dot is one logged event, placed at its Lamport time; hollow dots are APPLY events. An arrow is a message from its SEND to its RECV. Events on one line are ordered left to right.

Causally ordered pair 1. client-1 SEND Increment(x) at L=1 happened before replica-A APPLY x -> 1 at L=3. Chain: the SEND (L=1) is a message edge to replica-A RECV (L=2, received L=1), and that RECV is locally before the APPLY (L=3) on replica-A.

Causally ordered pair 2. client-1's first SEND (L=1) happened before client-3's first RECV GetReply(value=2) at L=20, although the two clients never talk to each other. Chain: the SEND reaches replica-A (RECV at L=2); every later event on replica-A follows it in local order, including its SEND GetReply at L=19; that SEND is a message edge to client-3's RECV (received L=19). This is why client-3 could read the value 2.

Concurrent pair. client-2 SEND Increment(y) at L=1 and client-1's last RECV at L=10. No chain of local order and message edges leads from one to the other: client-2 had received nothing before it sent, and replica-A received client-2's request only after it had already sent client-1's last reply. The Lamport values 1 and 10 differ, but they say nothing about which event came first: they are counters of two processes that had not yet exchanged a message. In wall-clock time the event with the smaller value (L=1) actually occurred later, because client-2 was started after client-1 had finished. The trace also shows equal values on different processes (all three clients log their first SEND at L=1), which is expected and carries no meaning.

Limitation of Lamport clocks. The clock rules are those of Lamport (1978). If a happened before b then L(a) < L(b), but the reverse does not hold, so two Lamport values cannot tell a causally ordered pair from a concurrent one. Vector clocks remove this: a happened before b exactly when the vector of a is smaller than the vector of b in every component, so concurrency can be detected from the timestamps alone.

Check of the clock rules against the trace (Task B1). In every RECV line the logged value equals max(previous value of that process, received value) + 1, and in every SEND and APPLY line it equals the previous value + 1. For example replica-A logs RECV at L=2 with received L=1, then APPLY at L=3 and SEND at L=4; client-1 then logs RECV at L=5 = max(1, 4) + 1.

7. Replication: what majority commit does and does not give (Task C1)

The partial-write anomaly. When only one replica acknowledges, the client reports failure, but that replica has already applied the delta and nothing rolls it back. Test T7 shows the client side of this (not committed, new_value = None). The replica side is visible in Figure 2 (Appendix D): in the third run replica A answered IncrementReply(new_value=15), so it had applied the delta, while the client printed FAILED not committed (replicas acked: 1/3). Increment applies the delta before it answers and no code path undoes it. The replicas now disagree, and a later Get that happens to hit that replica returns a value that was never committed. A consensus protocol such as Raft (Ongaro & Ousterhout, 2014) prevents this by separating storing from applying: the leader first appends the write to its log and replicates it, and an entry is applied to the state only after a majority has stored it. An entry that never reaches a majority is never applied and is later overwritten.

Consistency risk of Get. Get reads from the first replica that answers. A replica that was down during a committed write, or that has not received it yet, returns an older value, so a client can read stale data, and two reads answered by different replicas can go backwards. A majority read, or reading through a leader, would close this gap.

8. Known limitations

1. **No durability and no catch-up.** All state is in memory. A replica that restarts comes back empty, and nothing copies the writes it missed to it. If the replicas that hold a committed write are restarted, the write is lost. A persistent log that is replayed on start, and log replication to bring a lagging replica up to date, are the Week 6 mechanisms that address this.

2. **A failed write stays partially applied** (section 7). There is no rollback and no leader.

3. **The deduplication store grows without bound.** Keys are never removed, so memory grows with every operation. A real service would expire keys.

4. **Replicas are contacted one after another.** While one replica is down, every write waits for the full retry budget on it (0.2 + 0.4 + 0.8 s of pauses plus the failed connection attempts) before it returns. Test T6 takes several seconds for this reason.

5. **Test coverage gaps.** The Lamport rules are verified from the captured trace and not by an automated test. The crash scenario stops a server inside the test process instead of killing a separate operating-system process. The benchmark runs clients and replicas in one Python process, so the 16-client numbers include interpreter contention and overstate the latency of a real deployment.

6. **The B2 trace was not captured under real concurrency:** the three clients ran one after another.

9. Reflection questions

1. Why can the client not tell a lost request from a lost reply, and why is that harmless here? After a timeout the client knows only that no reply arrived before the deadline. That is equally true if the request never reached the server, if the server applied it and the reply was lost, and if the server is only slow. In the first case nothing was applied and in the second the delta already was, and no message can tell the client which. The design makes the difference irrelevant: the client resends with the same idempotency key, and the server either applies the operation for the first time or finds the key and returns the stored result. Either way the delta is applied once (T5, F3 and the fault log in section 4, where two RECV lines produce one APPLY line).

2. Why does the smaller Lamport value not mean the event happened first? In the trace client-2's SEND has L=1 and client-1's last RECV has L=10, and the two are concurrent. A Lamport value only counts the events its own process has seen so far, directly or through received messages. Client-2 had just started and had seen nothing, so its counter was still small; client-1 had already taken part in two request-reply exchanges. A smaller value is necessary for happening before, but it is not sufficient, and here the event with the smaller value was in fact the later one in real time.

3. Which failures are tolerated with three replicas and majority commit? The crash of any one replica: every committed write was acknowledged by at least two replicas, so it is still held by at least one live replica, and new writes still reach a majority (T6, F1). That is both safety (no committed write is lost) and liveness (writes keep committing). With two replicas down, liveness is lost, since no write can commit, but safety holds: the client reports failure and presents no value as committed (T7). The guarantee itself breaks when the replicas holding a committed write lose their memory, for example two replicas restarting one after the other, because state is not persisted and a restarted replica is not brought up to date. That is a safety violation.

4. Smallest change to survive restarts. Before acknowledging, each replica appends the applied operation (idempotency key, counter and resulting value) to a file and flushes it, and on start it replays that file to rebuild the counters and the deduplication store. This is a write-ahead log, and it anticipates the persistent replicated log of Raft (Ongaro & Ousterhout, 2014) from Week 6.

References

Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM, 21*(7), 558-565. https://doi.org/10.1145/359545.359563

Ongaro, D., & Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference* (pp. 305-319). USENIX Association.

Appendix A. Captured event log (Task B2), merged by Lamport time

Source files: `logs/client-1.log`, `logs/client-2.log`, `logs/client-3.log`, `logs/replica-A.log`. Lines with equal L are ordered by process name.

```
[client-1] SEND       Increment(counter=x, delta=1)                  L=1
[client-2] SEND       Increment(counter=y, delta=1)                  L=1
[client-3] SEND       Get(counter=x)                                 L=1
[replica-A] RECV      Increment(counter=x, delta=1)                  L=2   (received L=1)
[replica-A] APPLY     counter=x -> 1                                 L=3
[replica-A] SEND      IncrementReply(new_value=1)                    L=4
[client-1] RECV       IncrementReply(new_value=1)                    L=5   (received L=4)
[client-1] SEND       Increment(counter=x, delta=1)                  L=6
[replica-A] RECV      Increment(counter=x, delta=1)                  L=7   (received L=6)
[replica-A] APPLY     counter=x -> 2                                 L=8
[replica-A] SEND      IncrementReply(new_value=2)                    L=9
[client-1] RECV       IncrementReply(new_value=2)                    L=10   (received L=9)
[replica-A] RECV      Increment(counter=y, delta=1)                  L=10   (received L=1)
[replica-A] APPLY     counter=y -> 1                                 L=11
[replica-A] SEND      IncrementReply(new_value=1)                    L=12
[client-2] RECV       IncrementReply(new_value=1)                    L=13   (received L=12)
[client-2] SEND       Increment(counter=y, delta=1)                  L=14
[replica-A] RECV      Increment(counter=y, delta=1)                  L=15   (received L=14)
[replica-A] APPLY     counter=y -> 2                                 L=16
[replica-A] SEND      IncrementReply(new_value=2)                    L=17
[client-2] RECV       IncrementReply(new_value=2)                    L=18   (received L=17)
[replica-A] RECV      Get(counter=x)                                 L=18   (received L=1)
[replica-A] SEND      GetReply(value=2, found=True)                  L=19
[client-3] RECV       GetReply(value=2, found=True)                  L=20   (received L=19)
[client-3] SEND       Get(counter=x)                                 L=21
[replica-A] RECV      Get(counter=x)                                 L=22   (received L=21)
[replica-A] SEND      GetReply(value=2, found=True)                  L=23
[client-3] RECV       GetReply(value=2, found=True)                  L=24   (received L=23)
[client-3] SEND       Get(counter=x)                                 L=25
[replica-A] RECV      Get(counter=x)                                 L=26   (received L=25)
[replica-A] SEND      GetReply(value=2, found=True)                  L=27
[client-3] RECV       GetReply(value=2, found=True)                  L=28   (received L=27)
```

Appendix B. pip freeze

```
colorama==0.4.6
grpcio==1.84.0
grpcio-tools==1.84.0
iniconfig==2.3.1
packaging==26.3
pluggy==1.6.0
protobuf==7.36.2
Pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0
```

Appendix C. pytest output

```
============================= test session starts =============================
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\askha\PycharmProjects\DS_midterm\.ve
    nv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\askha\PycharmProjects\DS_midterm
collecting ... collected 11 items

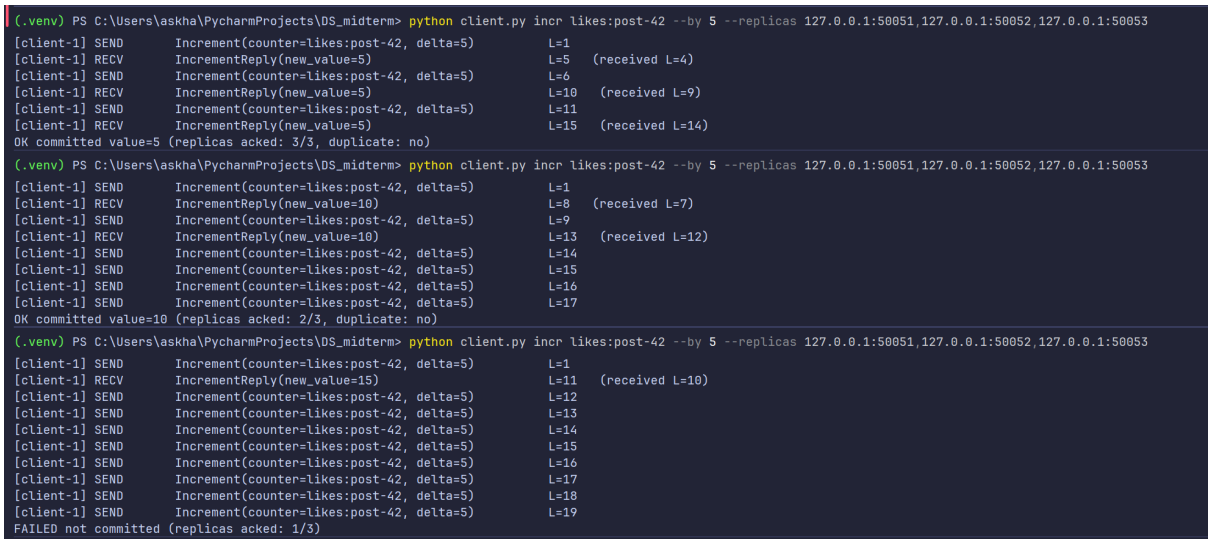
tests/test_counter.py::test_increment_applies_delta PASSED               [  9%]
tests/test_counter.py::test_duplicate_key_not_reapplied PASSED           [ 18%]
tests/test_counter.py::test_concurrent_increments_exact PASSED           [ 27%]
tests/test_counter.py::test_get_missing_counter PASSED                   [ 36%]
tests/test_counter.py::test_retry_after_timeout_is_safe PASSED           [ 45%]
tests/test_counter.py::test_majority_commit_two_acks PASSED              [ 54%]
tests/test_counter.py::test_no_commit_below_majority PASSED              [ 63%]
tests/test_counter.py::test_replicas_converge PASSED                     [ 72%]
tests/test_failures.py::test_replica_crash_mid_request_still_commits PASSED [ 81%]
tests/test_failures.py::test_duplicate_request_moves_value_once PASSED   [ 90%]
tests/test_failures.py::test_timeout_then_retry_moves_value_once PASSED  [100%]
============================= 11 passed in 16.20s =============================
```

Appendix D. Screenshots

The screenshots were taken on the test machine described in Table 2.

Figure 2

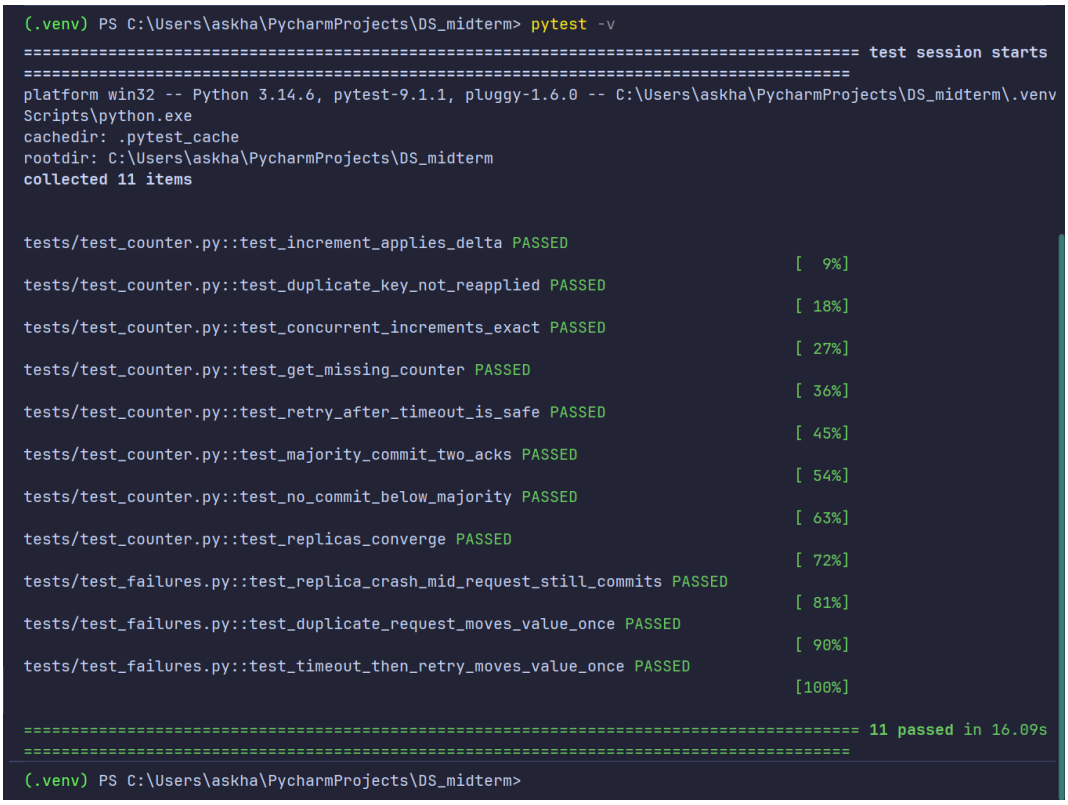
Quorum Write With Three, Two and One Replica Running



Note. First run: all three replicas acknowledge (3/3). Second run: replica C is stopped; the client tries it four times and the write commits with 2/3. Third run: replicas B and C are stopped; the client reports FAILED not committed (1/3), although replica A answered IncrementReply(new_value=15), which shows that it had applied the delta.

Figure 3

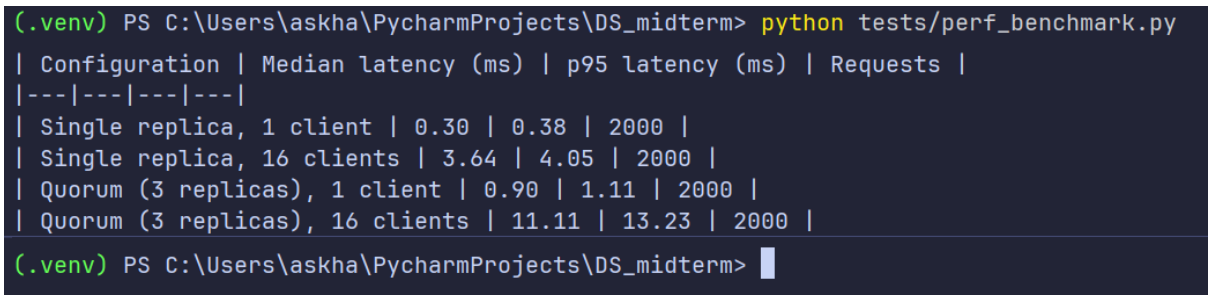
Test Suite Run With pytest -v



Note. All 11 tests pass. This is a repeat run; its duration (16.09 s) differs slightly from the run recorded in Table 3 and Appendix C (16.20 s).

Figure 4

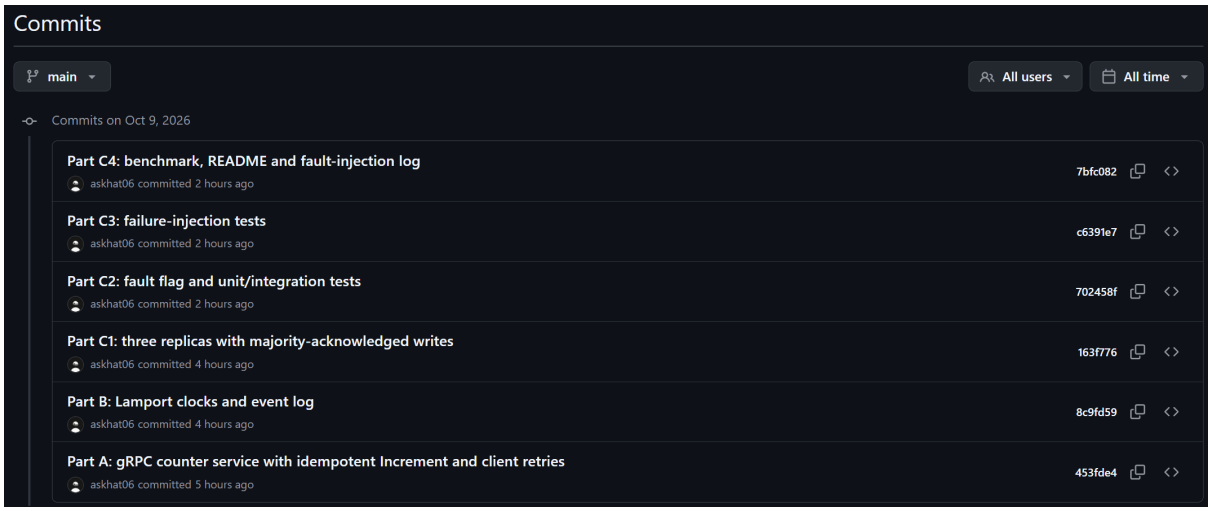
Output of tests/perf_benchmark.py



Note. A repeat run. The values differ slightly from Table 5, which reports the earlier run that was saved to a file; the ratio between the single-replica path and the quorum path is the same.

Figure 5

Commit History of the Repository on GitHub



Note. The six code commits, one per part of the assignment (A, B, C1, C2, C3, C4), made on 9 October 2026. The report was committed after this screenshot was taken.